Cybergymnastics Inc.

DITA Architect v0.5 — Full Team Codebase Review

Date: March 10, 2026

Participants:

Name	Title	Focus Area
Rafael Santos	Engineering Manager	Kickoff & Review Coordination
Anna Sidorova	Staff Engineer	Architecture & Data Flow
Jamie Okafor	Senior Frontend Engineer	Implementation Quality
Elena Vasquez	Principal Engineer	Code Quality & Type Safety
Maya Chen	UX Engineer	UX, Accessibility & Content Design
Marcus Wren	Senior DevOps Engineer	Build, Config & Infrastructure
Taylor Brooks	QA Lead	Test Coverage & Quality

Table of Contents

1. Rafael Santos — Kickoff Brief
2. Anna Sidorova — Architecture Review
3. Jamie Okafor — Implementation Review
4. Elena Vasquez — Code Quality Review
5. Maya Chen — UX Review
6. Marcus Wren — DevOps Review
7. Taylor Brooks — Test Coverage Review

Rafael Santos — Kickoff Brief

DITA Architect v0.5 Codebase Review Date: 2026-03-10 **Author:** Rafael Santos, Engineering Manager

Objective

We are doing a structured pre-v0.5 review of the entire DITA Architect codebase. This is an internal tool — security is explicitly out of scope. What is in scope: everything that would make this tool faster to work in, easier to extend, and less likely to surprise us during a demo. I want each of you to produce a written findings doc with severity ratings and concrete recommendations. We'll consolidate in two weeks.

Do not fix things during the review. Flag, document, discuss.

Current State Assessment

DITA Architect is a browser-based split-pane authoring tool: Lexical WYSIWYG editor on the left, Monaco XML editor on the right, bidirectional live sync, Heretto CMS integration, multi-tab file management, and tracked changes support. Here is what I read.

Architecture: The top-level wiring lives in `dita-architect.tsx`, which is the `ProfessionalDitaEditor` component. It coordinates four custom hooks — `useTabManager`, `useHerettoCms`, `useEditorUi`, and `useLocalFile` — and renders the entire application. `App.tsx` is a two-line shell that mounts the editor and drops in a `Toaster`. Clean entry point.

Sync layer: This is the most complex and most load-bearing part of the codebase. `SyncManager.tsx` orchestrates bidirectional sync between Lexical and Monaco using focus detection, a `syncTrigger` counter, and a `masterXmlRef` to anchor patches. `serializeLexicalToXml.ts` is ~1,000 lines implementing a surgical DOM-patching serializer: it clones the original document, walks recognized elements in parallel with Lexical nodes, and patches only what changed — preserving opaque DITA elements, processing instructions, and Heretto review markers. `parseXmlToLexical.ts` is the inverse: a recursive XML-to-Lexical tree builder with inline formatting, custom nodes (`DitaOpaqueNode`, `DitaCodeBlockNode`, `DitaImageNode`, `DitaPhRefNode`), and a `nodeOriginMap` side-table for round-trip fidelity.

Tabs: `useTabManager.ts` owns tab state — create, close (with unsaved-change guard), convert topic type, and switch. The `activeTab` derivation uses `tabs.find()` on every render, and `currentTopicType` runs a full XML parse on every `activeTab.xmlContent` change.

Heretto integration: `useHerettoCms.ts` (~577 lines) manages the entire Heretto lifecycle: connectivity check on mount, credential storage, folder navigation, debounced full-tree search (batched fetches with an in-memory folder cache), two-pass import verification, save with post-save read-back comparison, 30-second remote-change polling, and refresh. Abort controllers on saves, refreshes, and search.

Toolbar: `Toolbar.tsx` uses `window.prompt()` for URL input, inline `onMouseEnter` / `onMouseLeave` style handlers on every button (there are roughly fifteen of them), and toast notifications on every format action including undo and redo. The theme dropdown and the edit-mode accept/reject controls are also here. No `aria-` attributes anywhere in the file.

Editor config: The Lexical theme in `dita-architect.tsx` (lines 42-66) mixes Tailwind CSS utility classes with CSS variable references. The `h2` heading style is hardcoded (`text-2xl font-bold text-white mb-3`) and does not use a CSS variable, making it immune to theme switching. The `onError` handler for the Lexical editor is `console.error` — no error boundary escalation.

Review Focus Areas

Anna — Architecture

Focus: Component boundaries, data flow, and coupling between layers.

`dita-architect.tsx` is doing too much. It is the `LexicalComposer` mount, the entire modal rendering engine, the Heretto browser UI, the tab bar, the file drag-and-drop zone, and the top-level layout — all in one component. The `syncTrigger` counter pattern (an integer incremented to fire effects) is a workable hack but creates invisible coupling. The `nodeOriginMap` is a module-level mutable singleton shared across all tabs.

Jamie — Implementation Quality

Focus: Correctness risks, edge cases, and error handling.

The Heretto import verification flow fetches the file twice and compares the two responses for equality. The `herettoSearch` function uses a BFS traversal with an in-memory cache keyed by UUID that is never invalidated. The `_cachedXmlMeta` is module-level and shared across all editor instances. The `patchBodyChildren` function includes a `maxIterations` guard to prevent infinite loops.

Elena — Code Quality

Focus: Maintainability, type hygiene, and dead/duplicated code.

The `serializeInlineContent` function signature uses `any` for `lexicalNode`. `parseCellContent` is defined twice. Inline hover handlers appear on every single button. `useHerettoCms.ts` returns 44 values from a single hook.

Maya — UX and Accessibility

Focus: Usability, interaction design, and accessibility gaps.

`Toolbar.tsx` has zero `aria-label` attributes on icon-only buttons. The link insertion UX uses `window.prompt()`. The `Clear Contents` button has no guard — one misclick destroys the document.

Marcus — Build and DevOps

Focus: Build configuration, dependency health, and developer experience.

The `index.tsx` suppresses `ResizeObserver` errors globally. `@lexical/yjs` is in the dependency tree but not used. The `herettoFolderCache` is a module-level `Map` that grows unbounded.

Taylor — Test Coverage

Focus: Test surface identification and coverage gaps.

Priority testing targets: round-trip serialization fidelity, `patchInlineNodes` and `patchBodyChildren`, `convertDitaTopic`, `useHerettoCms` async flows, and `herettoSearch` BFS with cache.

Key Questions for the Team

1. **Anna and Jamie:** The `nodeOriginMap` is a global singleton. Is it actually safe across concurrent Lexical editor updates during a tab switch? What is the failure mode if it is not?
2. **Elena:** The `serializeLexicalToXml.ts` file is ~1,019 lines. Is this one file doing one thing well, or does it contain two or three distinct responsibilities that should be separated?
3. **Marcus:** Where does the Heretto API proxy live? Is it a Vite config, a separate Node server, or something else?
4. **Maya:** The `editMode` flag appears in `Toolbar`, `SyncManager`, `EditModePlugin`, and `TrackedChangesPlugin`. Is the interaction model for accept/reject edits clear to a first-time user?
5. **Taylor:** Given zero existing tests, where would a single day of test writing deliver the most risk reduction?

6. **Everyone:** The Lexical `onError` handler is `console.error`. What happens when Lexical throws during a sync operation?

---

v0.5 Versioning Goals

v0.5 is an internal milestone. The bar is: our team can use this tool daily without data loss, without surprising breakage, and without needing to know the implementation to work around its edges.

One constraint: I want at least the round-trip serialization path covered by tests before we tag v0.5. Everything else is negotiable. That one is not.

Two weeks. Written findings. No exceptions.

— Rafael

Anna Sidorova — Architecture Review

Date: 2026-03-10 **Reviewer:** Anna Sidorova, Staff Engineer, Cybergymnastics Inc. **Subject:** DITA Architect v0.x — Full Codebase Architectural Review **Scope:** All source files under `/Users/kirylrusetski/Documents/DITA Architect/` excluding `node_modules` and `dist`.

---

1. Codebase Inventory

Layer	Files
Entry	`index.tsx`, `App.tsx`
Root component	`dita-architect.tsx` (1,705 lines)
Custom hooks	`hooks/useTabManager.ts`, `hooks/useHerettoCms.ts`, `hooks/useEditorUi.ts`, `hooks/useLocalFile.ts`
Sync layer	`sync/index.ts`, `sync/parseXmlToLexical.ts` (354 lines), `sync/serializeLexicalToXml.ts` (1,019 lines), `sync/nodeOriginMap.ts`
Components	14 files: `SyncManager.tsx`, `Toolbar.tsx`, `BottomToolbar.tsx`, `EditModePlugin.tsx`, `TrackedChangesPlugin.tsx`, `MonacoDitaEditor.tsx`, `EmptyToH1Plugin.tsx`, `ShortdescPlugin.tsx`, `DitaOpaqueNode.tsx`, `DitaCodeBlockNode.tsx`, `DitaImageNode.tsx`, `DitaPhRefNode.tsx`, `TrackedDeletionNode.tsx`, `Tooltip.tsx`
Types	`types/tab.ts`, `types/heretto.ts`
Constants	`constants/dita.ts`, `constants/heretto.ts`
Lib utilities	`lib/xml-utils.ts`, `lib/heretto-utils.ts`
Other	`textAnalysis.ts`, `vite.config.ts`, `tsconfig.json`, `package.json`

---

2. Identified Invariants

1. **I-1 (Master XML is the ground truth).** `serializeLexicalToXml` always patches a fresh clone of `masterXmlRef.current`, not the previously-generated XML.
2. **I-2 (nodeOriginMap is always reset on parse).** `parseXmlToLexical` calls `nodeOriginMap.clear()` before populating it.
3. **I-3 (nodeOriginMap is module-global, not per-tab).** There is exactly one `nodeOriginMap` instance for the entire application.
4. **I-4 (XML cache in serializer is module-global).** `_cachedXmlSource` / `_cachedXmlMeta` are module-level singletons.
5. **I-5 (isSyncingFromXmlRef gates the onChange loop).** The `OnChangePlugin` callback must not fire when Lexical is being updated from XML.
6. **I-6 (savedXmlRef is mutated in place inside setTabs functional updaters).** Several places directly mutate `tab.savedXmlRef.current` inside a `setTabs` callback.

---

3. Identified Assumptions

1. **A-1 (Single active Lexical instance).** The multi-tab architecture renders all `LexicalComposer` instances simultaneously but hides inactive ones via `display: none`.
2. **A-2 (Vite dev server is always running during use).** The production build path has no equivalent server.
3. **A-3 (DOMParser is available and well-behaved).** Namespace handling and entity expansion are not explicitly tested.
4. **A-4 (DITA topics have a single root element).** Multi-root or wrapper documents are not handled.
5. **A-5 (Heretto REST API response format is stable).** Any API version change silently produces empty folder listings.
6. **A-6 (CSS Highlight API availability).** Correctly guarded with a capability check.
7. **A-7 (Tab IDs are stable for the lifetime of a tab).** Tabs are never re-keyed unless closed.

---

4. Numbered Findings

Finding 1 — [Critical] Module-Global `nodeOriginMap` Is Not Tab-Safe

The `nodeOriginMap` is a single `Map<string, NodeOrigin>` exported as a module singleton. Lexical node keys are scoped to a specific editor instance. When a second tab is opened, its `parseXmlToLexical` call clears the map and repopulates it. If any background `OnChangePlugin` event from another tab's editor fires between these two operations, the serializer will receive a corrupted or empty `nodeOriginMap`. The impact is that body element tags may be silently replaced by `<p>` tags on the next serialize, causing semantic data loss.

Finding 2 — [Critical] Module-Global XML Meta Cache Is Not Tab-Safe

The serializer maintains a module-level cache keyed on the raw XML string. If two tabs have identical raw XML, the cache will return meta derived from whichever was parsed first. The cache stores a parsed `Document` object indefinitely — a memory leak that grows with usage.

Finding 3 — [Major] `dita-architect.tsx` Is a 1,705-Line God Component

The render function spans lines 288-1,704. Even after four hooks were extracted, the component cannot be meaningfully tested in isolation.

Finding 4 — [Major] `useHerettoCms` Returns 39 Values

The consumer destructures all 39 and passes many directly into JSX. The hook also imports `formatXml` from `components/MonacoDitaEditor`, creating a hook-to-component dependency that inverts the expected layering.

Finding 5 — [Major] `MonacoDitaEditor.tsx` Is a Multi-Responsibility Module (868 Lines)

Contains: the Monaco React wrapper, a full XML tokenizer/formatter, the DITA content model, completion provider registration, auto-close-tag setup, a handwritten XML validator, and all Monaco syntax theme definitions.

Finding 6 — [Major] The Heretto Polling Loop Has No Backoff

A `setInterval` polls every 30 seconds for every tab with a `herettoFile`. No exponential backoff, no jitter, no circuit breaker. Each poll fetches full XML content.

Finding 7 — [Major] Toolbar Imports Types from Component Files

`ThemeName` and `SyntaxThemeName` type definitions should live in `types/`, not inside React component files.

Finding 8 — [Major] `ShortdescPlugin` Directly Mutates the Module-Global `nodeOriginMap`

A plugin component is responsible for maintaining sync-layer metadata, violating the map's invariant.

Finding 9 — [Minor] `useEditorUi` Owns Global Keyboard and Lifecycle Side-Effects

Finding 10 — [Minor] `TrackedChangesPlugin` Uses DOM Manipulation Outside React's Lifecycle

Finding 11 — [Minor] `herettoFolderCache` Is a Module-Global, Session-Persistent Map

Finding 12 — [Minor] `vite.config.ts` Contains Production-Impacting Business Logic

Finding 13 — [Minor] `Tab.savedXmlRef` Is a Mutable Ref Stored Inside Immutable React State

Finding 14 — [Minor] `textAnalysis.ts` Has No Type Exports and Lives at the Root

Finding 15 — [Minor] `parseXmlToLexical` Types All Lexical Nodes as `any`

---

5. Module Dependency Graph Issues

1. `lib/xml-utils.ts` -> `components/MonacoDitaEditor.tsx` (imports `formatXml`): a pure utility importing from a UI component.
2. `hooks/useHerettoCms.ts` -> `components/MonacoDitaEditor.tsx` (imports `formatXml`): a business logic hook importing from a UI component.
3. `hooks/useEditorUi.ts` -> `components/Toolbar.tsx` (imports `ThemeName`): a hook importing a type from a component.
4. `components/ShortdescPlugin.tsx` -> `sync/nodeOriginMap.ts`: a plugin writing to the sync layer's internal state.

---

6. Separation of Concerns Assessment

Concern	Current Location	Correct Location
`formatXml`	`components/MonacoDitaEditor.tsx`	`lib/xml-utils.ts` or `lib/xml-format.ts`
DITA content model	`components/MonacoDitaEditor.tsx`	`constants/dita.ts`
Theme type definitions	Component files	`types/editor.ts`
Monaco provider registration	`MonacoDitaEditor.tsx`	`lib/monaco-providers.ts`
`textAnalysis.ts`	Project root	`lib/text-analysis.ts`
`nodeOriginMap`	Module singleton	Per-editor-instance state
XML meta cache	Module singleton	Per-SyncManager instance

---

7. Recommended Refactoring Roadmap for v0.5

Phase 1: Correctness First

1. Extract `nodeOriginMap` from module scope into `SyncManager`-owned instance state.
2. Extract the XML meta cache from module scope into `SyncManager`.
3. Move `formatXml` to `lib/xml-format.ts`.

Phase 2: Type Safety 4. Define theme types in `types/editor.ts`. 5. Replace `any` types in sync layer with proper Lexical node types.

Phase 3: Decomposition 6. Decompose `MonacoDitaEditor.tsx` into at minimum four modules. 7. Extract modal components from `dita-architect.tsx` into `components/modals/`. 8. Reduce `useHerettoCms` return surface by

splitting into sub-hooks.

Phase 4: Reliability 9. Add backoff and per-tab polling limits to Heretto remote change detection. 10. Add TTL to `herettoFolderCache`. 11. Introduce integration tests for the sync round-trip.

Phase 5: Architecture (v1.0 Prerequisite) 12. Design a production backend for Heretto proxy and credentials.

Jamie Okafor — Implementation Review

Project: DITA Architect **Date reviewed:** 2026-03-10 **Files read:** All source in `components/`, `hooks/`, `sync/`, `lib/`, plus `dita-architect.tsx`, `types/`, `constants/`, `App.tsx`

The Big Picture First

This is a genuinely tricky domain — bidirectional sync between a Lexical visual editor and Monaco XML, with real CMS integration on top. A lot of this complexity is inherent to the problem space. That said, there's a meaningful gap between "hard because the domain is hard" and "hard because of how we wrote it." This review is about the latter.

---

1. The 1500-Line Monster (`dita-architect.tsx`)

`dita-architect.tsx` is 1704 lines long and is functionally a render function that contains seven modals worth of JSX inline. The hook extraction was the right call, and it got us pretty far — but the JSX is still a mess.

What I'd actually do: Create `components/modals/` with files for `ConfirmModal.tsx`, `HerettoStatusModal.tsx`, `HerettosBrowserModal.tsx`, `ImportVerificationModal.tsx`, `SaveModal.tsx`, `TopicTypeModal.tsx`. Then extract `<TopBar />`, `<TabBar />`, and `<HerettoContextBar />`. The main file should probably end up under 200 lines.

---

2. React Anti-Patterns

- **2a. `renderModal` inline function as component** — React sees the output as anonymous with no stable identity for reconciliation.
- **2b. `Tab` type stores refs inside plain state** — `savedXmlRef`, `monacoApiRef`, and `snapshotRef` are `{ current: ... }` objects living inside state. Fragile.
- **2c. Trigger counter pattern** — Five incrementing integer counters as "fire this effect" signals. Invisible and confusing.
- **2d. `activeTab` derived inline, not stable** — O(n) lookup on every render. `currentTopicType` below it parses XML with `DOMParser` on every render.
- **2e. `onLexicalChange` closure in `SyncManager`** — Stale closure vector.
- **2f. `doCloseTab` closes over `activeTabId`** — Can use stale value during rapid tab switching.

---

3. Lexical Plugin Patterns

- **3a. `EmptyToH1Plugin`** — Defensive double-check is correct but undocumented.
- **3b. `ShortdescPlugin` — global mutable singleton `nodeOriginMap`** — Scariest thing in the codebase. Multi-tab editor with shared node key space. Highest-priority bug risk.
- **3c. `TrackedChangesPlugin`** — Direct DOM mutation inside Lexical's managed root element.
- **3d. `TrackedChangesPlugin`** — O(n^2) LCS on every edit. No throttle or debounce.
- **3e. `SyncManager` safety timeout** — 200ms is arbitrary. Real race condition window for large documents.

---

4. Monaco Integration

- **4a. `validateDita` recaptured every render** — Stale closure risk.
- **4b. `providersRegistered` module-level flag** — Breaks on HMR.
- **4c. `setupAutoCloseTags` uses `any`** — No type safety on cursor manipulation.
- **4d. Auto-close tag logic doesn't handle attributes with `>`**

- **4e. `formatXml` in component file used as utility** — Inverted dependency.

---

5. Sync Layer

- **5a. Module-level cache in serializer** — Effectively useless in multi-tab scenarios.
- **5b. `parseXmlToLexical` uses `editor: any`** — Significant safety gap.
- **5c. `traverse` defined inside `updateFn`** — Recreated on every parse; could be module-level.
- **5d. `patchSectionFromHeading` is half-implemented** — May silently lose section body content.

---

6. Error Handling Gaps

- `herettoSearch` silently swallows all fetch errors
- `handleSaveConfirm` — `SecurityError` falls through to silent download
- `openHerettoStatus` fetch swallows all errors
- Heretto polling doesn't surface errors
- `handleHerettoSaveNew` — no abort controller

---

7. Performance Concerns

- All tabs render all the time (`display: none`) with full plugin stacks running
- `currentTopicType` parses XML on every `xmlContent` change
- `herettoFolderCache` never expires
- `BottomToolbar` runs Flesch-Kincaid calculation on every edit for all tabs

---

Summary: What to Actually Do, In Order

1. **Fix `nodeOriginMap` singleton** — scope it per editor instance
2. **Extract modals from `dita-architect.tsx`**
3. **Move `formatXml` to `lib/xml-utils.ts`**
4. **Replace hover handlers with CSS**
5. **Add error surfaces to Heretto polling and search**
6. **Move `herettoFolderCache` out of `constants/`**
7. **Document the trigger counter pattern**
8. **Replace `window.prompt()` in Toolbar**
9. **Add LRU cap to `herettoFolderCache`**
10. **Move refs out of Tab state**

Elena Vasquez — Code Quality Review

Project: DITA Architect **Stack:** React 19, Lexical 0.16, Monaco Editor, Vite 6, TypeScript 5.8 **Date:** 2026-03-10 **Reviewer:** Elena Vasquez, Principal Engineer

---

1. Type Safety

[Blocking] Pervasive `any` in the sync layer

`sync/parseXmlToLexical.ts` declares `editor: any` on line 21 and uses `any` throughout for Lexical node types. `sync/serializeLexicalToXml.ts` does the same. The serializer calls `.getChildren()`, `.getType()`, `.getTag()`, `.getCode()`, `.getLanguage()`, `.getRefType()`, `.getRefValue()` on untyped nodes with zero compile-time safety. A Lexical API change or a typo produces a silent runtime `undefined` that corrupts the XML output.

[Blocking] `any` in Monaco provider registration

`MonacoDitaEditor.tsx`: `provideDocumentFormattingEdits(model: any)` and `provideCompletionItems(model: any, position: any)`. Monaco exports typed interfaces for these callbacks.

[Blocking] `(window as any).showSaveFilePicker` and `(CSS as any).highlights`

Legitimate browser APIs with published TypeScript definitions available via `@types/wicg-file-system-access` and `@types/css-highlight-api`.

[Blocking] `target.savedXmlRef.current = content` — mutation inside `setTabs` updater

Directly mutates a property on a React state object inside a functional updater. Violates React's immutability contract.

[Non-Blocking] `nodeOriginMap` is a module-level singleton with no typing discipline

[Non-Blocking] `DitaTopicType` defined in two places — will drift

[Non-Blocking] Lexical node classes use public fields for private state — bypass Lexical's `getWritable()` versioning

---

2. Error Handling

[Blocking] `herettoSearch` silently discards folder fetch errors

Returns `[]` silently. Partial results with no indication of incompleteness.

[Blocking] `doHerettoRefresh` catches all errors and shows a generic toast

The `err` is fully discarded. A 403, a 500, a network timeout all produce identical feedback.

[Blocking] `handleHerettoOpen` double-fetches to "verify" — false confidence

Not a meaningful verification strategy. Both fetches hit the same endpoint with no cache-busting.

[Non-Blocking] `saveHerettoCredentials` merges two failure modes

[Non-Blocking] `EmptyToH1Plugin` — nested `update()` inside `registerUpdateListener`

[Non-Blocking] `SyncManager` — 200ms safety timeout and `registerUpdateListener` teardown race

---

3. Edge Cases

[Blocking] `nodeOriginMap` is never cleared when tabs are closed

[Blocking] `formatXml` is not idempotent on malformed XML — CDATA content silently truncated

[Blocking] `convertDitaTopic` — `XMLSerializer` injects spurious `xmlns=""` not stripped in this path

[Non-Blocking] `validateDitaXml` accepts `<topic>` but templates never create one

[Non-Blocking] `compareXml` tier-1 silently catches formatter exceptions

[Non-Blocking] Drag-and-drop `dragCounterRef` can underflow to negative

[Non-Blocking] `ShortdescPlugin` checks global `nodeOriginMap` — not multi-tab safe

---

4. Logic Correctness

[Blocking] `patchBodyChildren` — off-by-one bug on consecutive mismatches

When two consecutive type mismatches occur, `lexIdx` advances unconditionally, causing the second DOM child to be processed against the wrong Lexical node. Result: silent content corruption.

[Blocking] `serializeLexicalToXml` module-level cache — shared `Document` reference trap

[Blocking] `TrackedChangesPlugin` — deletion descriptor collision when text contains `::`

[Blocking] `activeTab` fallback to `tabs[0]` on stale `activeTabId`

[Non-Blocking] `doCloseTab` reads `activeTabId` from closure

[Non-Blocking] `SyncManager.tsx` — `syncTrigger` effect missing most dependencies

---

5. API Design

[Non-Blocking] Hook interfaces take `setTabs` directly — tight coupling to React state

[Non-Blocking] `useHerettoCms` returns 35 values from a single hook

[Non-Blocking] `SyncManager` and `EditModePlugin` overlap on edit mode ownership

[Non-Blocking] `renderModal` is a render function, not a component

---

6. Code Duplication

- `parseCellContent` implemented identically twice
- PI-adjacent whitespace filtering in three places (~50 lines duplicated)
- Dropdown outside-click pattern copy-pasted five times
- DOCTYPE and body-tag mapping duplicated across three files

---

7. Dead Code

- `DitaPhRefNode.__originalText` stored but never used for serialization
- `herettoSearchAbortRef` returned but never used by consumer
- `textAnalysis.ts` `readingLevel` computed but never displayed
- `TRACKED_CLASSES` constant never read
- `ClearEditorPlugin` provides no functionality in this application

---

8. Good Decisions

- The bidirectional sync architecture is thoughtfully designed — `masterXmlRef` prevents drift
- `serializeLexicalToXml` surgical patching strategy preserves PIs and unrecognized elements
- `compareXml` three-tier strategy is sensible
- `ImportVerificationState` is a well-typed discriminated union
- `useCallback` + `AbortController` pattern correctly prevents concurrent overwrites
- `TrackedDeletionNode.getTextContent()` returning `''` is exactly right
- Test coverage on pure utility functions is solid

Summary Verdict

The primary systemic problems are:

1. **The `any`-typed sync layer is a blocking correctness risk** — highest-priority fix.
2. **`savedXmlRef` mutation inside `setTabs` violates React's state immutability contract.**
3. **`nodeOriginMap` global singleton makes multi-tab correctness structurally impossible.**
4. **Two logic correctness bugs** — `patchBodyChildren` mismatch backtracking and TrackedChangesPlugin deletion descriptor collision — can produce silent XML corruption.

Fix the `any` types first. They are both the most impactful and the most tractable change.

Maya Chen — UX Review

DITA Architect | Full UI Audit | March 10, 2026

---

1. Content Design

Toolbar button labels and toasts

Every button fires a toast notification confirming what just happened. `Undo action performed`, `Redo action performed`, `Formatted as Heading 1`, etc. Toasts should communicate non-obvious outcomes, errors, or asynchronous results. Confirming `Toggled Bold` in a toast is redundant — bold text is immediately visible. Have you considered removing these confirmation toasts from synchronous formatting actions?

Clear Contents button

The `Trash2` icon fires a `toast.error('Cleared Editor Contents')`. Two problems: it uses `toast.error()` for a successful action, and there is no confirmation dialog. Closing a tab with unsaved changes shows a confirm modal, but clearing the entire editor clears without warning.

Edit Mode copy

`Enter Edit Mode` with tracked changes is powerful but explained nowhere. The toast `Entered Edit Mode — changes are isolated` is the only moment to set expectations. Have you considered `Review Mode active — additions are highlighted green, deletions shown with strikethrough`?

Menu duplication

Two menus with nearly identical contents: `DITA Architect` dropdown and `File Options` dropdown contain the same four operations with different label styles.

---

2. Accessibility

Missing aria-labels across the entire toolbar

Every button in `Toolbar.tsx` is icon-only with no `aria-label`. The Tooltip component is purely visual — no `role`, no `aria-*`, no keyboard trigger. Screen readers encounter unlabeled buttons containing SVGs with no accessible text.

19 specific unlabeled interactive elements identified across the toolbar, tab bar, and Heretto context bar.

Dropdown menus lack keyboard accessibility

None implement `role="menu"`, `role="menuitem"`, arrow key navigation, `Escape` to close, `aria-haspopup`, or `aria-expanded`. Keyboard-only users cannot operate core file management functions.

Modal focus management

Seven modals rendered as `fixed inset-0` overlays. None trap focus, return focus on close, have `role="dialog"`, `aria-modal`, or `aria-labelledby`.

Visual editor focus indicator

`outline-none` removes the default browser focus ring with no custom replacement. Keyboard users have no visual indication of focus.

Color-only state indicators

Tab dirty indicator is a small colored dot with no text label.

---

3. Empty States, Loading States, and Error States

- No welcome screen or feature tour on first launch
- No loading state for Lexical editor — flash of empty content during initial parse
- API failure toasts are consistent but minimal — no indication of transient vs credential vs server errors
- XML validation error messages are developer-level — could include suggested fixes
- Closing the last tab silently replaces content with no notification

---

4. Design System Consistency

- Two separate menus with identical items
- Hover states as inline event handlers instead of CSS classes (30-40 places)
- Toast severity usage inconsistent (`toast.error` for successful user actions)
- `AlertTriangle` icon overloaded for three distinct purposes
- Theme checkmark uses HTML entity instead of lucide-react icon
- `DitaOpaqueNode` fallback colors are dark-theme specific
- Tooltip positioning doesn't handle viewport boundaries

---

5. Suggested Copy Changes (28 specific suggestions)

Key changes:

- Remove all 10 synchronous formatting toast notifications
- `Clear Contents` -> add confirmation dialog, change from `toast.error` to `toast()`
- `Warning Note` -> `Note block`
- `Enter Edit Mode` -> `Enter Review Mode`
- `Commit` -> `Save to Heretto`
- `Heretto Status` -> `Heretto Connection`
- `Open Anyway` -> `Open with warnings`
- `Start typing your DITA content here...` -> `Start with a topic title, then press Enter to add a short description`

Summary of Highest-Priority Issues

1. No aria-labels on any toolbar buttons
2. Tooltips do not trigger on keyboard focus
3. No focus management in modals
4. `Clear Contents` fires without confirmation
5. Formatting action toasts are noisy and redundant
6. `Enter Edit Mode` feature is explained nowhere
7. `window.prompt` for Insert Link is jarring
8. Duplicate menus with inconsistent labels

Marcus Wren — DevOps Review

Date: 2026-03-10 **Project:** DITA Architect **Stack:** Vite 6, TypeScript 5.8, React 19

Build Status

`npx tsc --noEmit`

```
(no output)
```

Exit 0. Type check clean.

`npx vite build`

BLOCKED — Bash permission denied during review.

`npx vitest run`

BLOCKED — Bash permission denied during review.

Dependencies

```
{
  "name": "dita-architect",
  "private": true,
  "version": "0.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

Flags:

- `@anthropic-ai/claude-agent-sdk` in `devDependencies` — unusual placement. Needs clarification.
- Mixed pinning strategy: Lexical packages exact `0.16.0`, everything else uses ranges.
- No `test`, `lint`, or `format` npm scripts.
- No ESLint or Prettier config at all.
- `version: "0.0.0"` — not meaningful.

Config Review

`tsconfig.json`

Assessment: solid. `strict: true`, `isolatedModules: true`, `moduleResolution: "bundler"`, `noEmit: true`. Missing `noUnusedLocals` and `noUnusedParameters`.

`vite.config.ts`

No `build` section. No sourcemap config. Proxy is dev-only and conditional on `~/heretto.json` — correct by design. No `base` option.

`vitest.config.ts`

`globals: true` without matching tsconfig `types` entry. No `setupFiles` for jest-dom matchers. No coverage thresholds configured despite `@vitest/coverage-v8` being installed.

.gitignore

Assessment: adequate but not comprehensive.

Missing:

- `.env*` — **critical gap** for an auth-handling project
- `coverage/`
- `*.tsbuildinfo`

Missing Infrastructure

Item	Status
CI/CD	Absent
ESLint	Absent
Prettier	Absent
`test` npm script	Absent
`lint` npm script	Absent
`format` npm script	Absent
Coverage thresholds	Not configured
`setupFiles` for jest-dom	Not configured
`.env*` in .gitignore	Missing
`coverage/` in .gitignore	Missing
`build.sourcemap` in vite config	Not set

Recommendations (Priority Order)

1. Add `"test": "vitest run"` to `scripts`
2. Add `.env*` to `.gitignore`
3. Add `coverage/` to `.gitignore`
4. Set up ESLint
5. Add `setupFiles` for jest-dom matchers in vitest config
6. Add `noUnusedLocals: true` and `noUnusedParameters: true` to tsconfig
7. Clarify `@anthropic-ai/claude-agent-sdk` placement
8. Add a CI pipeline
9. Configure `build.sourcemap`

10. Add `"vitest/globals"` to tsconfig `types`

Taylor Brooks — Test Coverage Review

Project: DITA Architect **Review Date:** 2026-03-10 **Reviewer:** Taylor Brooks, QA Lead, Cybergymnastics Inc. **Vitest Version:** v4.0.18

---

1. Current Test Status

```
RUN  v4.0.18  /Users/kirylrusetski/Documents/DITA Architect

✓ tests/types.test.ts        (25 tests)   3ms
✓ tests/heretto-utils.test.ts (30 tests)  21ms
✓ tests/local-file.test.ts   (29 tests)   9ms
✓ tests/xml-utils.test.ts    (83 tests)  26ms
✓ tests/tab-manager.test.ts  (29 tests)  25ms

Test Files   5 passed (5)
      Tests  196 passed (196)
   Duration  719ms
```

196 tests. 5 files. All green.

---

2. Coverage Map

Libraries (`lib/`) — Well Covered

All exported functions from both `lib/` files are tested. `escapeXml` , `getTopicId` , `validateDitaXml` , `findUnrecognizedElements` , `compareXml` , `formatRelativeTime` , `convertDitaTopic` , `parseHerettoFolder` , `getFolderName` .

Types (`types/`) — Adequate

`createTab` exhaustively field-by-field covered. `resetTabIdCounter` used for test isolation.

Hooks (`hooks/`) — Partially Covered

2 of 4 hooks have any tests. Those tests exercise inlined copies of the logic, not the actual hook implementations.

Components (`components/`) — Zero Coverage

0 of 14 components have any tests.

Sync Layer (`sync/`) — Zero Coverage

- `sync/parseXmlToLexical.ts` — 354 lines. Zero tests.
- `sync/serializeLexicalToXml.ts` — 1019 lines. Zero tests.

This is the largest and most complex code in the project.

---

3. Critical Gaps

Gap 1 — `sync/serializeLexicalToXml.ts` — Zero coverage (CRITICAL)

1019 lines of patch-and-walk logic. Contains `patchInlineNodes`, `patchListElement`, `patchTableElement`, `patchBodyChildren`. None tested. Most likely to silently corrupt DITA content.

Gap 2 — `sync/parseXmlToLexical.ts` — Zero coverage (CRITICAL)

354 lines converting DITA XML into Lexical nodes. Whitespace normalization, element handling, inline formatting — all untested.

Gap 3 — `useHerettoCms.ts` — Zero coverage (HIGH)

10+ async functions: save, verify, refresh, search, poll. All untested.

Gap 4 — `TrackedChangesPlugin.tsx` — `diffWords` and `tokenize` untested (HIGH)

Full LCS-based word diff algorithm. Unexported, unreachable by current tests.

Gap 5 — `SyncManager.tsx` — Zero coverage (HIGH)

The runtime glue between Monaco and Lexical. All sync flag management untested.

Gap 6 — `useEditorUi.ts` — Zero coverage (MEDIUM)

Gap 7 — `useTabManager.ts` — Hook-level behavior untested (MEDIUM)

Gap 8 — Custom Lexical Nodes — Zero coverage (MEDIUM)

`DitaPhRefNode.getTextContent()` and `TrackedDeletionNode.getTextContent()` overrides are critical for serializer correctness. Never tested.

---

4. Test Quality Assessment

What the tests do well

- Behavioral focus in utility tests — test observable outputs, not internals
- Boundary conditions documented — empty strings, whitespace, malformed XML
- Counter isolation in `types.test.ts` — `resetTabIdCounter()` in `beforeEach`
- Reference independence between tabs tested

Problems

- `local-file.test.ts` tests inlined mirrors, not the actual code
- `tab-manager.test.ts` does not test `useTabManager` — misleading name
- No integration tests exist
- `@testing-library/react` and `@vitest/coverage-v8` installed but unused

---

5. Testing Strategy Recommendations for v0.5

1. Extract and directly test `diffWords` and `tokenize`
2. Write unit tests for sync layer without live Lexical editor — export pure helper functions
3. Write hook tests for `useTabManager` using `renderHook`
4. Write async tests for `useHerettoCms` with mocked `fetch`
5. Introduce integration-level roundtrip tests for XML -> Lexical -> XML
6. Rename `local-file.test.ts` and remove inlined mirror functions
7. Add localStorage persistence test for `useEditorUi`
8. Add coverage reporting to CI pipeline

Summary Verdict

The existing 196 tests are well-written for what they cover. The problem is perimeter: everything outside pure utility functions is completely unspecified. The sync layer represents the highest-value code in the application — it is the bidirectional bridge that makes the editor work — and it has zero tests.

If it's not tested, it doesn't work. Right now, the sync layer doesn't work — at least not as far as this test suite is concerned.

Cross-Team Findings Summary

Universal Critical Issues

These findings were independently identified by multiple team members:

Finding	Identified By	Severity
Module-global `nodeOriginMap` not tab-safe	Anna, Jamie, Elena, Rafael	**Critical**
Module-global XML meta cache not tab-safe	Anna, Jamie, Elena	**Critical**
`any` types throughout sync layer	Elena, Jamie, Anna	**Blocking**
`dita-architect.tsx` god component (1,705 lines)	Anna, Jamie	**Major**
`useHerettoCms` returns 35-39 values	Anna, Elena	**Major**
`formatXml` in component file (inverted dependency)	Anna, Jamie, Elena	**Major**
No aria-labels on toolbar buttons	Maya, Rafael	**Major**
Sync layer has zero test coverage	Taylor, Rafael	**Critical**
No CI/CD pipeline	Marcus	**Major**
`savedXmlRef` mutation inside state updaters	Elena, Jamie	**Blocking**
`patchBodyChildren` off-by-one bug	Elena	**Blocking**

Consolidated Priority Actions for v0.5

1. **Fix `nodeOriginMap` singleton** — scope per editor instance (Anna #1, Jamie #1)
2. **Fix `any` types in sync layer** — add Lexical type safety (Elena #1)
3. **Fix `patchBodyChildren` off-by-one** — silent content corruption (Elena #4)
4. **Add sync layer tests** — roundtrip fidelity (Taylor #1-2, Rafael mandate)
5. **Move XML meta cache to per-instance** — tab safety (Anna #2)
6. **Extract `formatXml` to `lib/`** — fix inverted dependency (Anna #3, Jamie #3)
7. **Add `aria-label` to all toolbar buttons** — accessibility (Maya #1)
8. **Add CI pipeline** — `tsc + vitest + vite build` (Marcus #8)
9. **Decompose `dita-architect.tsx`** — extract modals and sub-components (Jamie #2)
10. **Add `.env*` to `.gitignore`** — credential protection (Marcus #2)

This document was compiled from individual reviews submitted by the Cybergymnastics Inc. engineering team on March 10, 2026.